

An Efficient On-Device Federated Learning System through the Interplay of Client Selection and Batch Size with Watermarked Data

Tao Ling, Siping Shi, *Graduate Student Member, IEEE*, Hao Wang, *Member, IEEE*, Chuang Hu, and Dan Wang, *Senior Member, IEEE*

Abstract—Federated Learning (FL) enables edge devices to collaboratively train a global model using local data. However, the increasing prevalence of watermarks in datasets presents a new challenge to efficient FL. While watermarks assert data ownership and copyright, they introduce complexities that can lead to shortcut learning problems and mislead utility measurements for client selection. These issues are further exacerbated by batch size variations in efficient FL frameworks, ultimately undermining their time-to-accuracy performance.

We introduce *LotusFL*, an FL system designed to address the challenges posed by watermarked datasets in efficient FL. Specifically, it tackles the increased time-to-accuracy due to erroneous client selection and the accuracy degradation observed with larger batch sizes. *LotusFL* first estimates the characteristics of watermarks through statistical estimation and then adjusts the batch size using this estimated watermark information to balance the negative impact of the watermark against device idle waiting time. Additionally, its client selection mechanism, based on historical information, avoids the misleading utility signals from watermarks. This mechanism, working in conjunction with batch size adjustment, aims to accurately predict device runtime and identify potentially valuable devices.

We evaluated *LotusFL* through a real-world deployment on 40 edge devices. Compared to state-of-the-art efficient FL frameworks, *LotusFL* achieves superior performance, enhancing accuracy by up to 8.2% and reducing training time by 1.97 $\times$.

Index Terms—Federated learning, watermark, client selection, batch size, data and system heterogeneity, machine learning systems

I. INTRODUCTION

FEDERATED Learning (FL) has revolutionized the machine learning landscape by enabling model training on decentralized data while preserving user privacy. With the flourishing market for data-driven AI applications in edge computing, such as surveillance systems with intruder detection capabilities, the FL approach becomes increasingly pertinent. Here, data providers retain their raw video data on edge devices, participating in model training while mitigating privacy risks. Platforms like Flower [1] facilitate this type of

on-device FL, bridging the gap between centralized learning and data privacy.

However, as data becomes increasingly monetized, providers have grown concerned not only about the privacy of their information but also about its ownership and copyright protections. This shift has led to a surge in the adoption of watermarking technologies. Such watermarks are often embedded through transformations that may also introduce additional metadata into the raw data. For instance, many datasets use internet-sourced images bearing copyright watermarks. Google's SynthID [2] watermarks AI-generated images to combat misinformation. Furthermore, NexGuard's watermarking solution [3] is prevalent in the film industry for secure cinematic distribution.

All these scenarios raise a critical question: What is the impact of these watermarks on the performance of trained neural network models? Despite prior observations indicating that watermarked datasets can decrease the attentional focus of machine learning models [4]–[6], their impact within the realm of efficient FL remains unexplored. Our investigation goes a step further, encompassing both visible and invisible watermarks. Through an exhaustive measurement study, we have ascertained that the presence of watermarks can indeed adversely influence the model's attention.

Watermarks can introduce misleading patterns into the training data, which are often referred to as “shortcut features” [7], [8]. These shortcut features are superficial patterns that trained neural network models may learn instead of the underlying true patterns in the data, leading to a decline in model performance, a phenomenon known as the shortcut learning problem [9], [10]. Removing watermarks from the data is not trivial since watermarks are designed to be resilient to removal. Although existing machine learning models exhibit some robustness to shortcut features, efficient FL methods or systems can exacerbate the shortcut learning problem.

In this paper, we identify two critical issues exacerbated by watermarks in efficient FL: (i) Efficient FL tends to select clients whose data appears easier to train to save time or accelerate model convergence, a strategy employed in client selection [11]–[13]. Yet, watermarked data often displays superficial patterns that create an illusion of easy trainability, making such clients more likely to be selected by these algorithms. Conversely, data with richer and more complex features may be overlooked, even though training on such data could help mitigate the shortcut learning problem.

Tao Ling, Siping Shi and Dan Wang are with the Department of Computing, Hong Kong Polytechnic University, Kowloon, Hong Kong (e-mail: cstling@comp.polyu.edu.hk; si-ping.shi@connect.polyu.hk; cs-dwang@comp.polyu.edu.hk).

Hao Wang is with the Stevens Institute of Technology, Hoboken, USA (e-mail: hwang9@stevens.edu).

Chuang Hu is with the Department of Computer Science, Wuhan University, Wuhan 430072, China (e-mail: handc@whu.edu.cn).

Corresponding author: Dan Wang.

(ii) Efficient FL systems aim to enhance training efficiency by adjusting batch sizes during the training process [14], [15]. Increasing the batch size for laggard nodes can reduce average waiting times. However, as the batch size increases, watermark features can dominate the model's focus, obscuring the diverse, underlying patterns within the data. These two factors contribute to the resurgence of the shortcut learning problem in efficient FL systems.

We have observed and measured the negative impacts of watermarked datasets; however, designing a new federated learning mechanism to mitigate these effects presents two key challenges: First, it is often infeasible to know precisely how a watermark is embedded in the training data. This is due to the privacy requirements of FL, which prevent the FL workflow from accessing the processes involved in handling the original data, including the embedding of watermarks. Additionally, due to the heterogeneity of FL device environments, different types of watermarks must be considered.

Second, quantitatively determining the bias that watermarks introduce into client selection criteria and subsequently balancing the impact of these watermarks against training efficiency when adjusting batch sizes are hurdles.

In this paper, we develop *LotusFL* (Watermark-Agnostic on-device federated learning System through the interplay of client selection and batch size adjustment), an FL system for watermarked data that is agnostic to specific watermark embedding technologies and can tolerate watermark heterogeneity.

To address the first challenge, we observe that, intrinsically, watermarks can be characterized by their *location* and *intensity*, regardless of the specific embedding technologies. To maximally exploit such information, we develop an approach to estimate the location and intensity of the watermarks.

To address the second challenge, we propose a watermark-guided client selection method effective for non-IID data and non-IID watermarks. Specifically, in each iteration, the client selection algorithm selects clients that contribute the most to the loss function. Intuitively, clients with greater contributions to the loss lead to greater model improvements [16]. However, the loss function captures both genuine data contributions and errors introduced by watermarks. To separate these effects, we utilize historical information to distinguish between the data's contribution and the error from the watermarks, thereby informing the client selection process. Additionally, we employ adaptive batch sizes designed to reduce waiting times caused by straggler clients, thereby improving FL efficiency. The adaptive batch size parameter is adjusted to mitigate the influence of shortcut features introduced by watermarks.

We implemented a *LotusFL* prototype. *LotusFL* adopts Flower's parameter profile interface and enhances it with adaptive profiles and packages. *LotusFL* is available as a standalone package, and users can install it using a Python package manager such as pip. *LotusFL* supports machine learning tasks with watermarked datasets. Applications have the flexibility to customize various parameters within the profile file.

We evaluated *LotusFL* through experiments in a real-world setting by deploying it on a test network of 40 Jetson devices, each with varying computational capabilities. Compared with

existing efficient FL systems, *LotusFL* improves the time-to-accuracy and accuracy. We also conducted an ablation study of *LotusFL* to validate the contribution of each component to FL model performance on watermarked datasets. Notably, this is the first paper, to our knowledge, to set up FL efficiency experiments evaluating the non-IID degree of watermarks.

Our contributions can be summarized as follows:

- We investigate the scenario of FL where training data are watermarked. We demonstrate through experiments that watermarked data can significantly decrease model training accuracy. While we observe that watermarks can be considered as shortcut features in training data, we show that the performance of existing schemes is far from satisfactory (Sec. II).
- We develop *LotusFL*, a new FL system for watermarked training data. *LotusFL* effectively addresses two challenges: (i) lack of prior knowledge regarding watermarking technologies used on the data and (ii) the presence of non-IID watermarks across different clients (Sec. III).
- We implement a *LotusFL* prototype and evaluate it using a testbed of 40 Jetson devices with diverse hardware capacities. Our evaluations show that *LotusFL* outperforms existing baselines, improving accuracy by up to 8.2% and reducing training time by up to $1.97\times$ in the real-world testbed (Sec. IV).

II. BACKGROUND AND MOTIVATION

A. Background Technologies

Data watermarking is the process of embedding a message into an image. A message can be the identity of the owner, and the embedding process transforms this message into a watermark within the data. There are two primary types of watermarking technologies: spatial domain watermarking and frequency domain watermarking [17]. Spatial domain watermarking modifies image pixels (e.g., at specific locations) to encode the message, such as with the Least Significant Bit (LSB) technique [18]. Frequency domain watermarking typically involves transforming the image into a frequency domain and then embedding the message into specific frequency coefficients, using methods like Discrete Cosine Transform (DCT), Discrete Wavelet Transform (DWT), and Discrete Fourier Transform (DFT) [19].

However, when watermarks are widely used in datasets, they can cause *shortcut learning problems* in models trained on these data [7]. Several factors can lead to shortcut learning problems, such as insufficient model complexity or capacity, and the presence of repetitive, simple shortcut features in the dataset [8], [20], [21].

Federated Learning (FL) leverages a set of distributed clients $\mathcal{N} = \{1, \dots, N\}$ to iteratively learn a global model θ without requiring clients to share their raw data with the central aggregating server. Specifically, the FL training process aims to solve an optimization problem to minimize the loss of the global model:

$$\min_{\theta} F(\theta) := \sum_{i \in \mathcal{N}} p^{(i)} f_i(\theta), \quad (1)$$

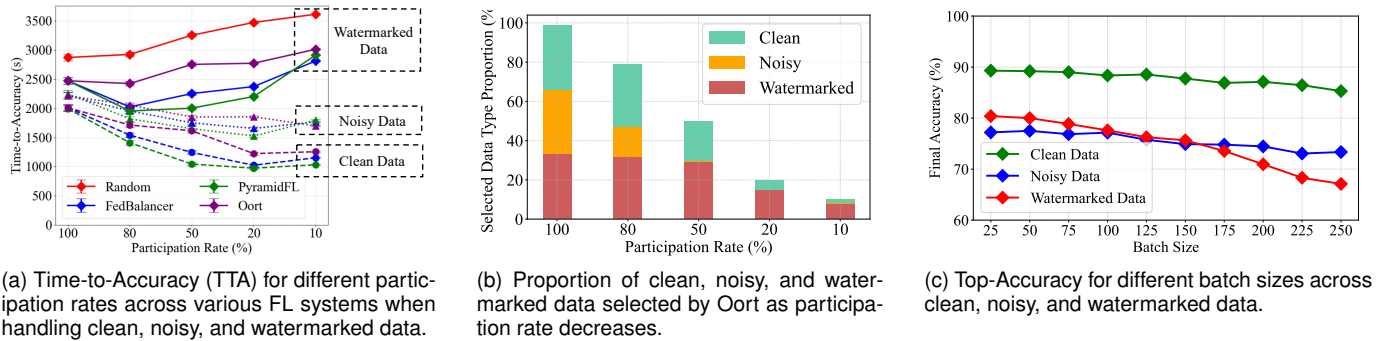


Fig. 1. Motivational experiments illustrating the challenges watermarked data introduces to federated learning efficiency and accuracy.

where $f_i(\boldsymbol{\theta}) = \frac{1}{|D^{(i)}|} \sum_{(x^{(i)}, y^{(i)}) \in D^{(i)}} \ell(x^{(i)}, y^{(i)}; \boldsymbol{\theta})$ is the local objective function of the i -th client with its local dataset $D^{(i)}$, and $p^{(i)} = |D^{(i)}| / \sum_{i \in \mathcal{N}} |D^{(i)}|$ is the relative data sample size. This is typically achieved through an iterative process where, in each round, participating clients minimize the loss on their local data to update their local models, and the server then aggregates these local model updates to refine the global model.

An important challenge in FL is that clients often possess non-IID data [22]. Intrinsically, non-IID data can lead to local models overfitting to specific data distributions, and this overfitting may propagate to global model.

B. Motivation

We designed two experiments to further investigate the impact of watermarks in FL and validate our motivation: (i) To determine whether the efficiency of model training in an FL system with client selection is affected by the client participation rate when using different types of datasets (clean, noisy, watermarked). (ii) To investigate the impact of batch size on model accuracy across these data types, particularly focusing on the performance with watermarked data compared to clean and noisy data.

Measurement setup. The VGG-16 model was employed for Federated Learning (FL), utilizing the CIFAR-10 dataset. Classification accuracy served as the performance metric for the FL system. Each client utilized the same watermarking method and parameters in these initial motivating experiments. For our evaluation, we selected several advanced efficiency-driven FL systems, including Oort [22], FedBalancer [23], and PyramidFL [24], and used random client selection as a baseline for comparison.

Measurement 1: Impact of participation rate on watermarked data handling and selection in FL systems. Fig. 1a shows the Time-to-Accuracy (TTA) for different participation rates across various FL systems when handling clean, noisy, and watermarked data. As the participation rate decreases, TTA generally decreases for clean and noisy datasets, aligning with expected efficiency improvements from client selection. However, for watermarked datasets, the TTA shows an opposite trend, indicating poorer performance and slower convergence to target accuracy.

To investigate this further, we examined how efficiency-driven FL systems select data types as participation rates change. Fig. 1b illustrates this using Oort on a dataset equally comprised of clean, noisy, and watermarked data. As client participation rates decrease from 100% to 10%, the results show that noisy data is discarded first, while the proportion of selected watermarked data increases significantly, ultimately reaching a ratio of approximately 8:2 compared to clean data. This supports our hypothesis that efficiency-driven FL systems tend to select watermarked datasets that appear easier to train due to superficial watermark features, thereby making the global model more susceptible to the shortcut learning problem.

Measurement 2: Impact of batch size on model accuracy for different data types. Fig. 1c illustrates the impact of varying batch sizes on model accuracy for different types of data using the CIFAR-10 dataset. As can be observed, increasing the batch size from 25 to 250 leads to a decline in accuracy for all data types. The accuracy drop for watermarked data is notably more significant, decreasing by 11.32%, compared to a 4.01% drop for clean data and a 3.85% drop for noisy data. We hypothesize that the additional accuracy decline in watermarked data is due to the increased proportion of watermark-related shortcut features within larger batches.

III. DESIGN

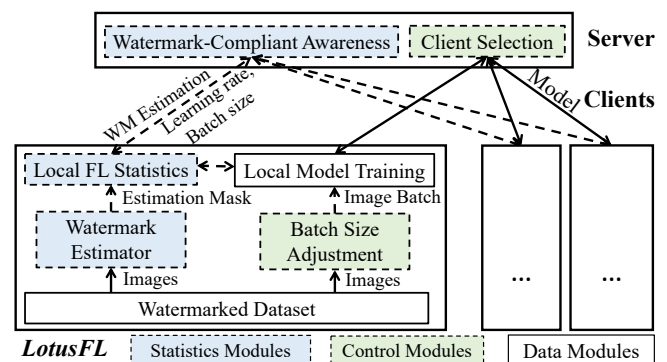


Fig. 2. Architecture of *LotusFL* Framework. Solid arrows indicate data flow, while dashed arrows indicate control information flow.

A. System Overview & Workflow

Fig. 2 shows the *LotusFL* architecture during a training round of the FL model. We added three modules to the basic FL framework, namely *Watermark Estimator* (Section III-C), *Batch Size Adaption* (Section III-E), and *Heterogeneity Aware Client Selection* (Section III-D).

The complete training process includes four steps. Our updated modules are divided into: *Statistics Modules* (blue), which are used to perceive watermarks and collect client performance statistics; and *Control Modules* (green), which adjust system-level control parameters using the information obtained from the statistics modules. The client preprocesses the data using the *Watermark Estimator*. This step involves individually inputting watermarked data to obtain the watermark estimation mask for the client's dataset. The *Local FL Statistics* module collects the estimation mask and gathers statistics on the client's computational performance, sending this information to the *Watermark-Compliant Awareness* module (Step 1). Local training using watermarked data first goes through the *Batch Size Adjustment* module to obtain a packaged image batch. This module automatically selects the appropriate batch size to accommodate the watermarked dataset and the client's computational performance, reducing the impact of watermarks in each image batch and minimizing additional waiting time caused by device performance imbalances (Step 2). After receiving utility metadata from clients, the server calculates utility and client performance. The *Client Selection* module selects clients with high utility and low bias, adjusting their batch size accordingly (Step 3). Finally, the server aggregates the local models from the selected clients to form a new global model for the next round of training (Step 4).

B. Problem Formulation

1) *Client Performance Modeling*: We refer to the client performance modeling in the study [14] and incorporate the computation process of the watermark estimator module mentioned in this paper. The modeling process involves several key steps. For each device with a fixed-size dataset, the number of iterations typically changes based on the batch size. Let κ^r represent the number of iterations during the r th epoch. The computation time, t_i^r , for device i can be defined as:

$$T_i^r = \kappa^r \frac{M \cdot b_i^r \cdot s}{C_i} + \nu_i + w_i, \quad (2)$$

Here, M refers to the model parameter size, and b_i^r is the batch size for device i at the r th epoch. C_i indicates the computational capacity of device i , typically the GPU or CPU frequency, while s denotes the CPU cycles required to compute the gradient of a single sample. The term ν_i accounts for the communication time, encompassing the duration for downloading the global model and uploading the local model, under the assumption that the model size M and the bandwidth for both downlink and uplink remain constant during training. Finally, w_i represents the computation time of local watermark estimation mask m (detail see in Sec. III-C), which depends on the data size, the number of data, and the computational load of the estimation algorithm which can be formulated as

$$w_i = d \cdot |D_i| \cdot E, \quad (3)$$

where d denotes the size of a single data sample (e.g., the number of pixels in an image), $|D_i|$ represents the size of the dataset on device i , and E indicates the complexity of the watermark estimation algorithm.

The epoch time T_i^r of the r th epoch on device i thus includes the computation time, communication time, and local watermark estimation mask computation time. Due to the synchronization barrier, faster devices must wait for the slowest one.

For device i , the waiting time can be expressed as $T_{\max}^r - T_i^r$, where $T_{\max}^r$ denotes the epoch duration of the slowest device during the r th epoch. The mean waiting time for all devices in the r th epoch is given by:

$$\mathcal{W}^r = \frac{1}{N} \sum_{i=1}^N (T_{\max}^r - T_i^r). \quad (4)$$

2) *Problem Formulation*: When considering FL training on watermarked datasets, we revise Eq (1) by incorporating a training efficiency constraint and watermark. With this modification, the problem can be reformulated as:

$$\min_{\theta} F(\theta) := \sum_{i \in \mathcal{N}} p^{(i)} f_i'(\theta) \quad (5a)$$

$$\text{s.t.} \begin{cases} \sum_{i \in \mathcal{N}} p^{(i)} = 1, & (5a) \\ T^C = \sum_{r=1}^R T_{\max}^r \leq T^B & (5b) \end{cases}$$

$$\mathcal{W}^r \leq \epsilon, \quad \forall r \quad (5c)$$

$$b_i^r \in \mathbf{Z}^*, \quad \forall r, i \quad (5d)$$

where $f_i'(\theta) = \frac{1}{|D^{(i)}|} \sum_{(x^{(i)}, y^{(i)}) \in D^{(i)}} \ell(WM(x^{(i)}, n_k^{(i)}), y^{(i)}; \theta)$ represents the local objective function of the i -th client with a watermarked dataset. The function $WM(\cdot)$ denotes the watermark embedding function, where each data sample $x_k^{(i)}$ is embedded with a digital watermark $n_k^{(i)}$. Eq. (5a) represents the normalization constraint for $p^{(i)}$. These weights are determined by the client selection algorithm, which incorporates each client's utility function $U(i)$ (see Sec. III-D). Eq. (5b) defines the constraint on the average waiting time, where R represents the total number of epochs, and T^C (the completion time) refers to the duration taken to finish these R epochs. The completion time T^C must remain within the time budget T^B due to constraints such as the limited computational capacity of edge devices. Eq. (5c) ensures that the average waiting time per epoch is kept minimal, with ϵ being a small positive value close to zero.

C. MaxEnt-based Watermark Estimator

In FL environments, the watermarking techniques used by each client can vary and are often unknown to the server. This variability renders many existing watermark estimation methods, which rely on specific watermarking techniques, ineffective. Despite these differences, watermarks generated by various techniques often share statistical characteristics, such as common locations or patterns. To address this variability while leveraging these shared characteristics, we employ a

Maximum Entropy (MaxEnt) estimation approach, inspired by the work in [25], [26]. This method does not require the use of prior distributions to estimate the watermark under uncertainty.

We adopt the classical watermarking model [27], where the embedding of a watermark can be modeled as $x' = x + n$, where x' is the watermarked image, x is the original image, and n represents the watermark. We aim to estimate the watermark n from the watermarked image x' .

Assuming the watermark can be modeled as a Gaussian random variable, we define the watermark sample $n_{u,v}$ and the image sample $x_{u,v}$ on a grid of size $M \times M$, where $1 \leq u, v \leq M$. Under the assumption that all samples are independent and identically distributed, given $n_{u,v}$, the conditional probability density of $x_{u,v}$ is:

$$p_{n_{u,v}}(x_{u,v} | n_{u,v}) = \frac{1}{\sqrt{(2\pi\sigma_{n_{u,v}}^2)^N}} \exp \left\{ -\frac{\Delta_n^T \Delta_n}{2\sigma_{n_{u,v}}^2} \right\}, \quad (6)$$

where the $\sigma_{n_{u,v}}$ of the watermark in the (u, v) location signifies its *intensity*, $\Delta_n = x_{u,v} - n_{u,v}$. Higher variance indicates a more noticeable watermark (albeit with possible image distortion), whereas lower variance results in a subtler watermark.

To estimate the watermark across the entire image, we introduce a *local estimation mask* $\mathbf{m} = [\hat{n}_{u,v}]_{1 \leq u, v \leq M}$, which is a matrix capturing each client's watermark information in the local dataset. The index (u, v) in this mask indicates the watermark *location*. Each $\hat{n}_{u,v}$ is determined by maximizing the entropy of the estimated watermark distribution subject to the observed data constraints. The optimization problem can be formulated as follows:

$$\begin{aligned} \hat{n}_{u,v} = \operatorname{argmax}_{\tilde{n}_{u,v}} & \left(-\sum_{u,v} p(\tilde{n}_{u,v}) \log p(\tilde{n}_{u,v}) \right) \\ \text{s.t.} & \sum_{u,v} p(\tilde{n}_{u,v}) (x_{u,v} - \tilde{n}_{u,v})^2 = \sigma_{n_{u,v}}^2 \end{aligned} \quad (7)$$

where $\tilde{n}_{u,v}$ represents a hypothetical watermark value being considered during the optimization process. The objective function maximizes the entropy of the estimated watermark distribution, ensuring that the distribution is as uniform as possible given the constraints. The constraint ensures that the estimated watermark values align with the observed data by matching the variance $\sigma_{n_{u,v}}^2$.

Compared to the MAP-based watermark estimation method used in the study [27], [28], we chose to use the MaxEnt criterion in our design for this paper. The reason is that the MAP criterion requires statistical prior knowledge, such as the number of samples and a consistent dataset for each iteration. In our study, we use dynamic batch sizes and different client selections in each round.

D. Watermark-Driven Client Selection Algorithm.

We employ a utility-based client selection that initially seemed promising. This type of approach dominates in system-level efficient FL [22], [29], [30].

However, as shown in Sec. II-B, client selection under a watermarked dataset did not achieve the expected performance. Upon further investigation, we found that utility-based

methods rely on importance estimates of the data to select appropriate clients. The calculation of these importance metrics is derived from performance indicators during model training, such as loss. However, the watermark-induced shortcut feature could inflate importance estimates due to artificial loss in model training from regularization artifacts, leading to higher bias across clients [31].

To correct this, our algorithm adapts by designing a more robust utility assessment. This adjustment aims to patch the initial utility approach, ensuring that our selection is based on accurate data contributions rather than distorted watermarked shortcut feature effects.

1) Adaptive Client Selection: We briefly summarize the design principles of mainstream client selection methods. The general challenges of FL are usually data heterogeneity and device heterogeneity. Measuring the utilities of FL clients and then scheduling clients based on their utility measures is the core idea behind FL client selection algorithms. In each training round, each client is measured for its utility/priority and the clients with the best utility measurements are selected for model training and aggregation.

Utility-based selection criteria: An overall utility function for a client i are often represented by its statistical utility and system utility as $Util(i) = Util_{\text{stat}}(i) \times Util_{\text{sys}}(i)$, which represents the usefulness of a client's local update to the global model, and measure the degree of client's straggling or overhead. A general utility function is:

$$U(i) = |D^{(i)}| \sqrt{\frac{1}{|D^{(i)}|} \sum_{k \in D^{(i)}} Loss(x_k)^2} \times \left(\frac{T}{t_i}\right)^{\kappa(T < t_i) \times \nu}, \quad (8)$$

leveraging the client's utility in terms of the duration of each training round. For statistical utility, $Loss(x_k)$ denotes the training loss for sample x_k . In training period, prioritizing clients with larger losses is essential, as their samples contribute more significantly to learning gains. Global system utility equals 1 (i.e., no punishment) for non-stragglers and increases exponentially for stragglers. It is a soft deadline to penalize stragglers where t_i is the client duration in training round, τ is the exponent controlling the penalty for stragglers, T is the developer-preferred duration of each round.

Watermark-aware selection criteria. Although we identify the most "useful" clients or data through the training loss with respect to the current model weights, there are clients that appear to have high utility but are misled by watermarked data. In FL training, the use of watermarked data introduces superficial shortcut features that mislead the model. As a result, the model fails to learn the essential data characteristics and consistently performs poorly on unseen test data, yielding higher loss values. The degraded model performance causes the utility-based selection criteria to continue selecting watermarked data, thereby entering a "vicious cycle".

Therefore, we propose a watermark-aware selection method for the second round of client selection. The core insight is that both watermarked samples and high-contributing samples can result in large losses, yet they can be distinguished by examining historical changes. Losses caused by watermarked samples exhibit smaller variations between iterations compared to those

from high-contributing samples [22]. As training progresses, a well-performing model gradually learns the characteristics of high-contributing samples, leading to a reduction in the losses they cause. In contrast, since no meaningful features can be learned from watermarked data, it maintains a high level of loss.

In each round, the historical statistic loss computed by each client is stored and used to calculate the degree of change in the current round's loss relative to the loss in previous rounds. The loss difference of the current round is accumulated with the loss differences of historical rounds, applying decaying weights. Finally, our utility function is :

$$U(i) = |D^{(i)}| \sqrt{\frac{1}{|D^{(i)}|} \sum_{k \in D^{(i)}} (\Delta Loss_{hist}^{(i)})^2} \times \left(\frac{T}{t_i}\right)^{\mu(T < t_i) \times \nu}, \quad (9)$$

where the $\Delta Loss_{hist}^{(i)}$ can be expanded to:

$$\Delta Loss_{hist}^{(i)} = \sum_{r=1}^R \frac{1}{1+r} \cdot \Delta Loss(x_k)_{R-r}^{(i)}. \quad (10)$$

In here the older rounds' loss differences are assigned smaller weights. The computational complexity is $O(R \times N)$, and hence it does not introduce additional overhead.

2) *Algorithm Design*: The Alg. 1 describes our FL system server process including three selection criteria. For utility aware selection part, the algorithm conceptualizes client selection as a multi-armed bandit problem, with each client representing an arm and their data utility serving as the reward. In each round r , selection criteria first update each client's utility based on the latest feedback (Line 4). Then it takes advantage of this updated information to focus on clients that have previously demonstrated high utility, thereby refining the client pool $\mathcal{E}$. At the same time, the algorithm allocates a fraction of the selection capacity (denoted by ζ) to explore new clients (Lines 6-13). This exploration is not random but is guided by the system speed of untried clients ($N - \mathcal{E}$), leveraging device model information to infer potential performance. Additionally, due to this exploration mechanism, clients that were excluded in early training rounds due to relatively poor quality data or watermarked data still maintain opportunities for selection in subsequent rounds. As ζ approaches 1, the algorithm shifts towards complete exploration. We consider the straggler in heterogeneous device situation and use the global waiting time information $\mathcal{T}$ to penalty straggler utility (Line 7). Smaller waiting time devices have a lesser penalty, and the degree is tuned by straggler penalty τ . At the same time, the penalty term will also affect the batch size. A higher penalty term will result in a larger batch size to speed up the training process (Line 7). The interplay between client selection and batch size is reflected in Line 12, implemented through a more fine-grained selection strategy, which is further detailed in Sec. III-F.

E. Batch Size Adjustment for Heterogeneous Device

The reasons for introducing the batch size adjustment mechanism are: (i) As observed in Fig. 1c, raising the batch size slightly enhances the negative effects of the watermark's

Algorithm 1: Utility and Watermark Aware Client Selection

Data: Client set $\mathcal{N}$, sample size $\mathcal{K}$, round time set $\mathcal{T}$, explored clients set $\mathcal{E}$, last involved round $\mathcal{L}$;

Parameter: Exploitation factor ζ , straggler tolerance ϵ , straggler penalty τ ;

Result: Client set $\mathcal{P}$

```

1 Initialize:  $\mathcal{P} \leftarrow \emptyset$ ;  $r \leftarrow 0$ ;
2 while Selection condition is met do
3    $r \leftarrow r + 1$ ;
4   UpdateWithFeedback( $\mathcal{T}, \mathcal{E}, \mathcal{L}$ );
5   foreach client  $i \in \mathcal{E}$  do
6      $U(i) \leftarrow U(i) + \sqrt{\frac{0.1 \log r}{\mathcal{L}(i)}}$ ;
7      $U(i) \leftarrow U(i) \times \left(\frac{\text{avg}(\mathcal{T})}{\mathcal{T}(i)}\right)^\tau$ ;
8     Receive  $T_i^r$ ;
9   end
10   $U(\cdot) \leftarrow \text{SortAsc}(U(\cdot))$ ;
11   $\mathcal{P} \leftarrow \text{SampleByUtil}(U(1 - \zeta) \times \mathcal{K})$ ;
12   $\mathcal{P} \leftarrow \mathcal{P} \cup \text{InterplayBatchSize}(\mathcal{P}, \epsilon, r, \mathcal{T}, T^r)$ ;
13   $\mathcal{P} \leftarrow \mathcal{P} \cup \text{SampleBySpeed}(\mathcal{N} - \mathcal{E}, \zeta \times \mathcal{K})$ ;
14 end
15 return  $\mathcal{P}$ ;
```

shortcut feature on the client's model. However, scaling the batch size can alleviate the burden on devices with weaker performance, creating a trade-off with the negative impact of the watermark; (ii) In edge computing, the heterogeneous computation capacities of devices cause significant synchronization barriers, leading to inefficiencies in model training. Dynamic batch size adjustment can reduce waiting time, thereby improving overall training efficiency.

1) *Batch Size and Watermark Influence Trade-off*: When adjusting the batch size, it is also necessary to balance the trade-off between the performance gain and the reduction in the negative impact of the watermark. Through the watermark estimator, we can gauge the degree of watermarking, which, to some extent, reflects data quality. The research [32] finds that the order of data quality can be used to adjust the learning rate to achieve a higher convergence rate. the order of dirty data means if we have two datasets, one is the dirty data, and the other is clean data(or relatively clean). The model learns clean data first($\text{order}[\text{clean}, \text{dirty}]$) or dirty data($\text{order}[\text{dirty}, \text{clean}]$) first. The relationship between dirty data order and learning rate can be established as in Lemma 1.

Lemma 1. λ_f and D_f represent the learning rate and data size for the first dataset, respectively. λ_s represents the learning rate for the second dataset. G denotes the total data size, including both the first dataset and the second dataset. Γ_f and Γ_s are the noise estimates for the first and second datasets, respectively. Then the optimal learning rate λ_s can be calculated by minimizing $B(\lambda_f, \lambda_s)$, expressed as:

$$B(\lambda_f, \lambda_s) = \frac{4\Gamma_f^2 \left(\frac{D_f}{G}\right)^{2\lambda_s-1} \lambda_f^2}{G(2\lambda_f-1)} + \frac{4\Gamma_s^2 \left(1 - \left(\frac{D_f}{G}\right)^{2\lambda_s-1}\right) \lambda_s^2}{G(2\lambda_s-1)}. \quad (11)$$

Algorithm 2: Batch Size Adjustment

Input : $b_l^{r+1}, b^r, \kappa^{r+1}, c_i, \nu_i, \epsilon, \bar{\mathbf{m}}_r, \bar{\mathbf{m}}_{r+1}, \lambda^r$
Output: b^{r+1}, λ^{r+1}

```

1 Initialize  $b^{r+1}$  as an array;
2 for client  $i = 1$  to  $N$  do
3   Calculate  $T_i^r$  and Upload to Server;
4   Receive  $\hat{T}_l^{r+1}, \kappa^{r+1}$  from Server;
5   if  $i == l$  then
6     Assign  $b_i^{r+1} = b_l^{r+1}$ ;
7   end
8   else
9     Estimate  $\hat{b}_i^{r+1} = \left( \frac{\hat{T}_i^{r+1} - \nu_i - w_i}{c_i} \right)$ ;
10    Compute
         $\xi \leftarrow \left( \frac{\hat{T}_i^{r+1} - \frac{\epsilon}{2} - \nu_i - w_i}{\kappa^{r+1} \cdot c_i} - \hat{b}_i^{r+1} \right) / \left( b_i^r - \hat{b}_i^{r+1} \right)$ ;
11    Adjust  $b_i^{r+1} \leftarrow \left\lfloor b_i^r \cdot \xi + \hat{b}_i^{r+1} \cdot (1 - \xi) \right\rfloor$ ;
12    Solve  $\lambda_i^{r+1} \leftarrow \operatorname{argmin}_{\lambda_i^{r+1}} B(\lambda_i^r, \lambda_i^{r+1})$ ;
13  end
14 end
```

Now under our algorithm, the optimized learning rate for the training of the next batch can be formulated as

$$\operatorname{argmin}_{\lambda^{r+1}} B(\lambda^r, \lambda^{r+1}) = \frac{4|\bar{\mathbf{m}}_r|^2 \left(\frac{b^{r+1}}{b^r + b^{r+1}} \right)^{2\lambda^{r+1}-1} (\lambda^r)^2}{(b^r + b^{r+1})(2\lambda^r - 1)} + \frac{4|\bar{\mathbf{m}}_{r+1}|^2 \left(1 - \left(\frac{b^{r+1}}{b^r + b^{r+1}} \right)^{2\lambda^{r+1}-1} \right) (\lambda^{r+1})^2}{(b^r + b^{r+1})(2\lambda^{r+1} - 1)} \quad (12)$$

where the λ^r is given. Now we can calculate the optimal learning rate for the r th batch training with watermark estimation $|\bar{\mathbf{m}}^{r+1}|$, batch size b^{r+1} in $(r+1)$ th batch, watermark estimation $|\bar{\mathbf{m}}^r|$, and the learning rate λ^r , batch size b^r in the r th batch.

2) *Algorithm Design:* We designed Alg. 2 to run on the client side, aiming to address Eq. 8 by adjusting the batch size and learning rate. In this algorithm, the client receives global information from the server and calculates the optimal batch size locally. The specific steps of the algorithm are as follows: After initialization, the active clients perform one round of training, during which they record and upload the current training time. Upon receiving the training time information, the server identifies the slowest client as l and sends the estimated next round's slowest training time $\hat{T}_l^{r+1}$ to each client (Lines 3-4). Consequently, for the slowest client l , the batch size remains unchanged (Line 6). For other clients, the algorithm estimates the batch size required to match the slowest training time in the next round (Line 9). However, this estimated batch size, being a theoretical optimal value, cannot be directly applied due to real-world fluctuations. Therefore, we incorporate a buffering mechanism to scale the batch size. The scaling factor ξ is dynamically adjusted based on the difference between the estimated batch size and the current batch size, ultimately computing the scaled batch size for the next round b_i^{r+1} (Lines 10-11). Finally, the new batch size is

used to adjust the corresponding learning rate λ_i^{r+1} according to Lemma 1, and this minimization problem is solved using the SciPy library (Line 12).

F. Interplay of Client Selection and Batch Size

In Alg. 2, we implemented batch size adjustment to reduce average waiting time and mitigate the negative impact of watermarks in small batches. The utility-based selection criteria include system utility, which penalizes stragglers by monitoring computation time, thereby reducing additional waiting time. Although these two methods appear to address the same issue from different angles, the inefficient devices they optimize do not overlap and can complement each other's shortcomings to achieve higher efficiency gains.

Inefficient devices are those whose training time is significantly longer than the average training time in a round of FL training. There are generally two types of inefficient devices: those with low computational performance, often limited by CPU or GPU capabilities, or even power constraints, and those with excessively large datasets. The imbalance in data quantity among devices is also a classic problem in FL. The client selection method typically discards inefficient devices directly. While effective, this approach loses the valuable data contained in devices with large datasets, which could be beneficial for FL training. On the other hand, the batch size adjustment method can include devices with valuable large datasets but does not directly speed up devices with low computational performance; instead, it makes other devices accommodate them. Although this reduces waiting time, it presents inefficiency issues in the long run.

Our idea is to achieve higher efficiency gains through the interplay of client selection and batch size adjustment. The specific steps are as follows: the client selection algorithm first identifies inefficient devices and classifies them as pending devices. The batch size adjustment algorithm then calculates the expected batch size for the pending devices and estimates the computation time with the new batch size. If the estimated time exceeds the straggler tolerance, the device is discarded; otherwise, it continues training.

1) *Algorithm Design:* In the Interplay with Batch Size phase, we integrate the server-side content from Alg. 1 to complete the global information calculation related to batch size adjustment in Alg. 3. Additionally, we combine batch size adjustment with the criteria for filtering inefficient devices, retaining potentially valuable clients in the client set $\mathcal{P}$. We first identify the slowest device l and estimate the possible number of iterations κ^{r+1} and the round time $\hat{T}_l^{r+1}$ for the next round (Lines 2-4). Next, using the straggler tolerance ϵ , we identify potentially valuable clients that were excluded by the initial client selection and add them to the client set $\mathcal{P}$ (Line 7). Finally, we send the time predictions for the next round to the clients for local batch size calculation (Line 10).

IV. IMPLEMENTATION & EVALUATION

A. Prototype Implementation

We implemented *LotusFL* in Python 3.8 and employed PyTorch 1.6 to implement the FL workflow and our proprietary modules. *LotusFL* manages metadata, such as watermark

Algorithm 3: InterplayBatchSize($\mathcal{P}, \epsilon, r, \mathcal{T}, T^r$)

Output: Additional client set $\mathcal{P}_{\text{new}}$

```

1 Initialize:  $\mathcal{P}_{\text{new}} \leftarrow \emptyset$ ;
2  $l \leftarrow \arg \min_{i \in \mathcal{P}} T_i^r$ ;
3 Calculate  $\kappa^{r+1} \leftarrow \left\lceil \frac{|D_l|}{b_l^{r+1}} \right\rceil$ ;
4 Estimate the  $(r+1)$ -th time:  $\hat{T}_l^{r+1} \leftarrow \kappa^{r+1} \cdot T_l^r$ ;
5 foreach client  $i \notin \mathcal{P}$  do
6   if  $\frac{\hat{T}_i^{r+1}}{b_i^{r+1}} < \epsilon$  then
7      $\mathcal{P}_{\text{new}} \leftarrow \mathcal{P}_{\text{new}} \cup \{i\}$ ;
8   end
9 end
10 Send  $\hat{T}_l^{r+1}, \kappa^{r+1}$  to client  $l$ ;
11 return  $\mathcal{P}_{\text{new}}$ ;

```

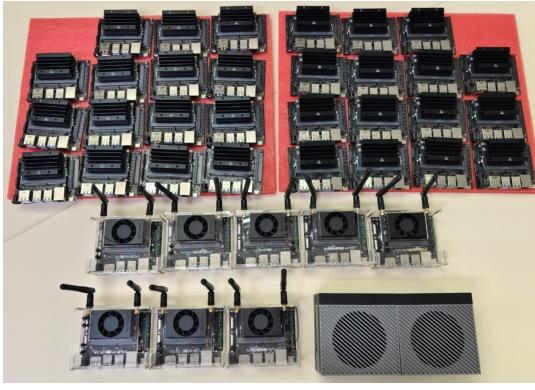


Fig. 3. The *LotusFL* testbed setup in our lab.

estimation masks, by backing them up to persistent storage and dynamically retaining or deleting them as needed to minimize storage requirements. The implementation is open-sourced¹.

LotusFL extends Flower [1], an on-device FL system. This allows *LotusFL* to leverage Flower's existing on-device framework and compatibility features. *LotusFL* adopts Flower's parameter profile interface and enhances it with adaptive profiles and packages. Additionally, *LotusFL* integrates Ansible [33] to enable batch configuration of clients. *LotusFL* is available as a standalone package, and users can install it using a Python package manager such as pip.

B. Experimental Setup

TABLE I
PERFORMANCE COMPARISON OF EDGE DEVICES

Device	GPU (Compute)	CPU (Cores, Freq.)	Memory
AGX Orin	248 TOPS	8-core, 2.2 GHz	32GB
Orin Nano	40 TOPS	6-core, 1.5 GHz	8GB
Orin Nano*	20 TOPS	6-core, 1.5 GHz	4GB
Nano	472 GFLOPS	4-core, 1.43 GHz	4GB

We assessed the performance of our system in both a client-server simulation environment and a physical testbed that we

constructed. The simulation environment was implemented on a workstation server equipped with an Nvidia RTX 3090 GPU and an AMD Ryzen 9 5900X CPU, running Ubuntu 20.04 LTS, simulating 10 to 100 clients by default. The testbed environment mirrored this server-side setup. For client devices in the testbed, we employed 2 NVIDIA Jetson AGX Orin, 3 Jetson Orin Nano 8GB, 5 Jetson Orin Nano 4GB, and 30 Jetson Nano devices. Their performance details are provided in Table I. The FL communication component utilized the 'torch.distributed' library. Simulation setup involves a central server Python process and multiple client Python processes to emulate the FL environment.

1) *Models and Datasets:* In our experimental phase, we employed a range of DNN for image classification tasks. The first was a Lite-CNN, characterized by its simplicity yet effectiveness. This model comprised two convolutional layers followed by 3x3 max pooling layers. The network also included two fully connected dense layers, culminating in a softmax output layer for classification. Alongside this custom CNN, we integrated two well-established models: VGG-16 and ResNet-18. The initial learning rate was set to 0.1, and the default batch size was 64.

In our experiments, we assessed the effectiveness of our proposed approach using several well-known image classification datasets: MNIST, CIFAR-10, Tiny-ImageNet [34], and COVID-FL [35]. Notably, the COVID-FL dataset inherently contains images with various real-world watermarks. For model selection, we utilized Lite-CNN for MNIST, VGG-16 for CIFAR-10, and ResNet-18 for both Tiny-ImageNet and COVID-FL.

2) *Watermark Settings:* To construct our watermarked datasets, we referred to the methodology in [28] and utilized a variety of watermark embedding techniques. These included frequency-based methods such as Discrete Wavelet Transform (DWT), Discrete Cosine Transform (DCT), and Discrete Fourier Transform (DFT), as well as spatial domain techniques like Least Significant Bit (LSB) manipulation and explicit (visible) watermarking [17], [36]. For each clean dataset (MNIST, CIFAR-10, and Tiny-ImageNet), we simulated watermarked versions using these techniques. The proportion of data processed by each watermarking method was controlled by a watermark heterogeneity parameter β , allowing us to simulate non-IID watermarks across clients. As mentioned, the COVID-FL dataset was used with its original, diverse real-world watermarks.

Although the parameters defining watermark intensity differ among methods (e.g., intensity in explicit watermarks refers to transparency, while in some frequency domain techniques like DFT, it pertains to modulation amplitude), we standardized the representation of intensity for all watermarks on a scale from 0 (no watermark) to 1 (maximum intensity). In spatial domain watermarks, the embedding location denotes the position within the image (such as the center or edges). In frequency domain watermarks, it refers to the frequency bands targeted within the image spectrum.

3) *Benchmark Methods:* We compared *LotusFL* with the following peer FL designs when dealing with watermarked data:

¹<https://anonymous.4open.science/r/LotusFL>

TABLE II
FL METHODS PERFORMANCE COMPARISON UNDER DIFFERENT SETTINGS.

Setting	Clean Dataset & Inference				Watermarked Dataset & Clean Inference				Watermarked Dataset & Inference			
	MNIST	CIFAR-10	Tiny.	COVID-FL	MNIST	CIFAR-10	Tiny.	COVID-FL	MNIST	CIFAR-10	Tiny.	COVID-FL
FedAvg	.961±.000	.855±.004	.305±.043	.838±.015	.943±.006	.695±.024	.263±.079	.658±.030	.905±.009	.546±.003	.140±.036	.596±.096
Oort	.966±.000	.872±.004	.374±.017	.852±.012	.948±.012	.697±.024	.278±.036	.679±.027	.930±.003	.563±.018	.182±.027	.598±.030
FedBalancer	.979±.000	.856±.007	.395±.013	.873±.010	.950±.009	.709±.012	.324±.018	.683±.042	.926±.006	.564±.027	.203±.018	.607±.027
PyramidFL	.981±.000	.864±.005	.415±.009	.881±.013	.968±.006	.748±.030	.336±.015	.695±.024	.933±.003	.671±.036	.230±.109	.668±.036
LotusFL	.978±.000	.860±.007	.381±.019	.885±.021	.975±.009	.833±.091	.370±.024	.719±.030	.937±.003	.757±.036	.312±.033	.678±.079

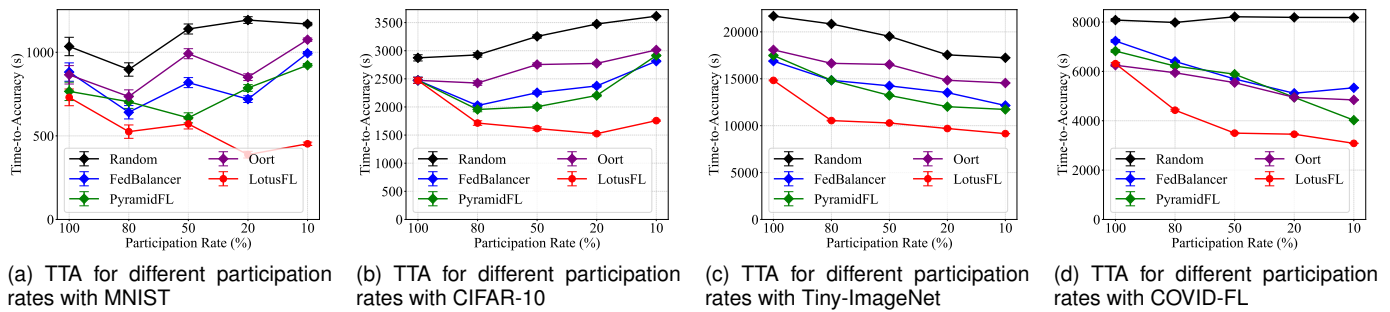


Fig. 4. Time to Accuracy (TTA) for different participation rates using the benchmark method across various datasets.

- FedAvg [37]: Used to establish a performance baseline. Its client selection mechanism is random.
- Oort [22]: An FL client selection framework considering both statistical utility and system efficiency. We applied it with hyperparameters as specified in the original paper: penalty factor $\alpha = 2$, $\varepsilon = 5$, and an initial exploration factor of 0.9.
- FedBalancer [23]: A client selection framework that prioritizes clients with more "informative" data. We used hyperparameters as per the original paper: $w = 20$, $lss = 0.1$, $dss = 0.05$, $p = 1$.
- PyramidFL [24]: A fine-grained client selection framework. We configured it with hyperparameters from the original paper, where the dropout ratio ranges from $d_0 = 0.1$ to $d_1 = 0.6$.

4) *Evaluation Metrics*: Our performance evaluation focuses on two aspects: FL performance and utility. FL performance is assessed using accuracy, which measures the percentage of correct predictions by the FL models on a distributed dataset, and loss, indicating the prediction error with lower values signifying better performance. For utility, the key metric is speedup, which evaluates the efficiency of the model's training process. Speedup is defined as the reduction in time taken for the FL models to converge compared to a baseline, reflecting the training efficiency in the FL environment. These metrics are widely used in the evaluation of efficient FL methods [22]–[24].

C. Results & Analysis

1) *Overall performance*: Our experimental results showcase two types of performance metrics: the final model accuracy, representing the model's usability, as shown in Table II, and the time-to-accuracy, representing the training efficiency. The target accuracy is set to the final accuracy of FedAvg.

As detailed in Table II, our experimental setup examines three distinct scenarios: (i) both training and inference using clean datasets, (ii) training with watermarked datasets but performing inference on clean data, and (iii) both training and inference using watermarked datasets. The settings for data and watermark heterogeneity are set to $\alpha = 0.5$, $\beta = 0.5$, as defined in Sec. IV-C2.

In the clean inference scenario, our approach achieves an accuracy of 97.5% on MNIST, 83.3% on CIFAR-10, 37.0% on Tiny-ImageNet (denoted as Tiny.), and 71.9% on COVID-FL. These results are significantly higher than those obtained with other methods, such as *FedAvg*, *Oort*, *FedBalancer*, and *PyramidFL*. The improvement is even more pronounced in the watermarked dataset & inference scenario, with scores of 93.7% on MNIST, 75.7% on CIFAR-10, 31.2% on Tiny-ImageNet, and 67.8% on COVID-FL.

For benchmarking training efficiency, we compare the time-to-accuracy under different client participation rates, with the training setup using watermarked dataset and watermarked inference. The final results are shown in Fig. 4. Compared to the best alternative methods, *LotusFL* saves time under a 10% participation rate by 57.9% on MNIST, 41.7% on CIFAR-10, 37.1% on Tiny-ImageNet, and 36.3% on COVID-FL. The results indicate that *LotusFL* achieves lower time-to-accuracy across almost all participation rates.

The underperformance of other methods can be attributed to their inability to effectively address the dual challenge posed by non-IID data and the presence of watermarks. Methods like *FedAvg* and *Oort*, while foundational in FL, lack specific mechanisms to counteract the nuanced effects of watermarked data, leading to compromised accuracy. *FedBalancer* and *PyramidFL*, despite introducing robustness in client selection, do not directly tackle the issue of shortcut learning induced by watermarks.

TABLE III
ACCURACY COMPARISON FOR DIFFERENT DATASETS ON VARYING NON-IID DEGREES.

Dataset	$\frac{\alpha}{\beta}$	PyramidFL			LotusFL		
		100.0	5.0	0.5	100.0	5.0	0.5
MNIST	0.5	.957	.935	.933	.959	.944	.937
	10.0	.955	.948	.942	.964	.958	.946
	100.0	.968	.960	.953	.970	.971	.964
	w/o	.978	.964	.961	.978	.966	.963
CIFAR-10	0.5	.711	.695	.671	.782	.772	.757
	10.0	.768	.709	.676	.812	.797	.799
	100.0	.822	.762	.678	.828	.799	.794
	w/o	.836	.769	.748	.832	.830	.797
Tiny.	0.5	.272	.235	.230	.328	.316	.312
	10.0	.343	.283	.273	.351	.340	.329
	100.0	.355	.319	.326	.370	.343	.346
	w/o	.376	.382	.331	.386	.392	.374

2) *Effect of LotusFL in Non-IID data and Non-IID Watermark*: We referred to [28] and defined the degree of non-IID data and non-IID watermark as follows:

In a multi-client training scenario, each client's data is independently sampled with class labels from N classes, following a categorical distribution with vector q ($q_i \geq 0$, $i \in [1, N]$, $\|q\|_1 = 1$). Non-IID client data is simulated by sampling q from a Dirichlet distribution, $\text{Dir}(\alpha\mathbf{p})$, where $\mathbf{p}$ is the prior class distribution, and $\alpha > 0$ determines client similarity. An infinite α implies uniform client distributions, while α near zero results in maximum divergence among clients.

For the non-IID watermark degree, it denotes the variance in watermark patterns among clients, defined by intensity (I) and location (L). Watermark characteristics vary across clients with gradient profile I, L_β , where lower β clients have less intense, peripherally located watermarks, and higher β clients have more intense, centrally located watermarks.

In our experimental analysis, the combined impact of non-IID data and non-IID watermark on the LotusFL system is depicted through Table III, revealing a compounded decrease in accuracy with the simultaneous presence of both non-IID conditions. At the same time, we also included training on clean datasets (marked as w/o) for comparison. With α of 0.5 and β of 0.5, the accuracy in CIFAR-10 drops to around 16.5% compared with the clean dataset. Modifications to the system, as reflected in the second tables, show improvements in this challenging scenario, with a notable increase in accuracy. Under the same high non-IID conditions, the accuracy improves to approximately 73.1%. The third tables, which focus on the percentage of improvement, highlight the effectiveness of our modifications. In scenarios with non-IID data and watermark, our method achieves a substantial improvement, with the most pronounced increase in accuracy reaching up to 9.46%.

TABLE IV
ABLATION STUDY - WATERMARK ESTIMATION COMPONENT

Setting	MNIST	CIFAR-10	Tiny	COVID-FL
Stacking	.932±.002	.542±.010	.201±.008	.621±.011
MAP	.966±.007	.716±.008	.303±.012	.655±.010
MaxEnt	.937±.003	.757±.036	.312±.033	.678±.079

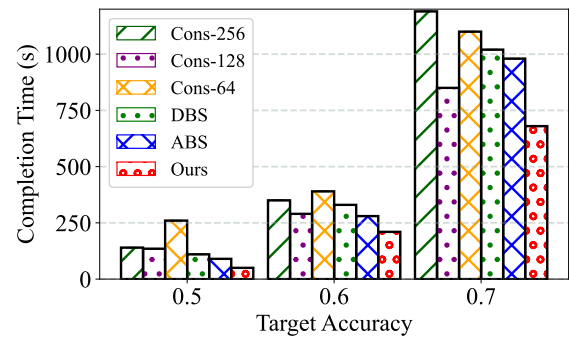


Fig. 5. TTA of different batch size adjustment component

TABLE V
ABLATION STUDY - BATCH SIZE ADJUSTMENT COMPONENT

Setting	MNIST	CIFAR-10	Tiny	COVID-FL
DBS	.962±.003	.734±.009	.315±.011	.663±.010
ABS	.969±.002	.763±.009	.342±.007	.672±.009
Ours	.937±.003	.757±.036	.312±.033	.678±.079

TABLE VI
ABLATION STUDY - CLIENT SELECTION COMPONENT

Setting	MNIST	CIFAR-10	Tiny	COVID-FL
Loss	.929±.005	.576±.009	.241±.005	.628±.010
Mod. Div.	.932±.002	.751±.009	.308±.010	.664±.011
HistLoss	.937±.003	.757±.036	.312±.033	.678±.079

3) *Ablation Study*: We analyze three components designed for such environments: watermark estimation (Stacking and MAP), batch size adjustment (ABS and DBS), and client selection (Loss and Mod. Div.) in Tables IV, V, and VI.

Study on watermark estimation. Our exploration delved into the efficacy of MaxEnt-based watermark estimation by comparing it against both its variants and analogous statistical methodologies. One is the strategy of stacking all dataset images to generate a uniformly weighted mask (denoted as Stacking). Another is the MAP-based estimation method [38]. As shown in Table IV, the MaxEnt approach manifested a notably superior accuracy enhancement relative to its counterparts, with a 4.1% increment over MAP within the CIFAR-10 dataset.

Study on batch size adjustment. We evaluated batch size adjustment by comparing our method against Adaptive Batch Size (ABS) [14], Dynamic Batch Size (DBS) [15], and Consistent batch size (Cons) based on time-to-accuracy. The results are shown in Fig. 5. Table V shows our approach performs better on the CIFAR-10 dataset, which has strong heterogeneity and fewer images. Conversely, for larger, more uniform datasets like Tiny-ImageNet and COVID-FL, the performance difference from DBS is minimal. This is because batch size adjustment primarily aims to provide a global setting that helps clients with less data find a more suitable batch size. Therefore, if client local datasets are sufficiently large, the improvement from this adjustment may be marginal.

TABLE VII
SPEEDUP AND ACCURACY ON DIFFERENT CHOICES OF *LotusFL* PARAMETERS.

Parameter		ζ		ϵ			τ		
Value		2	5	30	50	70	0.40	0.60	0.80
MNIST	Accuracy	.956±.000	.957±.000	.957±.000	.957±.000	.965±.000	.959±.000	.957±.000	.969±.000
	Speedup	1.52±0.07	1.44±0.04	1.29±0.04	1.44±0.04	1.47±0.04	1.44±0.04	1.30±0.04	1.28±0.04
CIFAR-10	Accuracy	.783±.010	.801±.011	.799±.011	.801±.011	.801±.012	.801±.011	.793±.010	.805±.013
	Speedup	2.15±0.23	2.12±.010	1.87±0.09	2.12±.010	1.91±0.11	2.12±0.10	1.79±.010	1.52±0.09
COVID-FL	Accuracy	.732±0.10	.750±0.11	.742±0.09	.750±0.11	.747±0.11	.750±0.11	.757±0.11	.761±0.15
	Speedup	1.67±.04	1.65±.04	1.60±.04	1.65±.04	1.64±.04	1.65±.04	1.62±.04	1.61±.04

Study on client selection. In our ablation study focusing on client selection, we maintained identical inputs for the estimation mask while employing different selection criteria. We compared Loss-based selection [22], Model Divergence (Mod. Div.) [39], and ours Historical Loss (HistLoss). Insights from Table VI reveal that the enhancements attributed to HistLoss are predominantly observed in datasets with smaller capacities, such as CIFAR-10, and in datasets where the watermark patterns are relatively fixed, such as COVID-FL. It is also observed that, although other forms of selection criteria exhibit limited improvements over the baseline, their compatibility with watermark estimation is not as pronounced.

4) *Parameter Sensitivity Analysis:* In the analysis of parameter sensitivity for the Utility and Watermark Aware Participant Selection Algorithm, we concentrate on the exploitation factor (ζ), straggler tolerance (ϵ), and straggler penalty (τ). The evaluation involves keeping each parameter constant while varying the others to discern their individual impacts.

In our experiments with ζ on the MNIST dataset, a value of 2 leads to a $1.52\times$ speedup, while increasing accuracy to 95.6%. This indicates that a lower ζ enhances efficiency, while a higher ζ improves accuracy, as a higher ζ attempts to incorporate data from previously unseen clients. This implies additional time required for exploration but also helps in reducing accuracy loss due to the watermark effect from individual clients. Regarding ϵ , the algorithm maintains consistent accuracy across its range. On MNIST, altering ϵ from 30 to 70 keeps the accuracy level around 95.7% to 96.5%. Although theoretically, a lower ϵ could mitigate the misclassified clients with rich data, thus enhancing accuracy, we found that the actual setting of ϵ needs to be tailored to the dataset and use case. Notably, a value of 50 is recommended in the absence of prior knowledge. The impact of τ is particularly significant in terms of speedup, especially evident in the CIFAR-10 dataset. With $\tau = 0.40$, *LotusFL* reaches about $2.12\times$ speedup, the highest among the values tested. This underscores the parameter's role in mitigating delays brought about by slower clients in the learning process.

5) *Robustness under Adversarial Conditions:* We evaluated the robustness of *LotusFL* under adversarial conditions by introducing three types of perturbations: dynamic watermarks, universal adversarial samples, and label noise. The result is shown in Tab. VIII. In our experiments, 25% of the clients acted adversarially by contributing corrupted data to the training process.

For dynamic watermarks, certain clients altered the intensity and location of their watermarks at specified rounds within a limited variation range. To generate universal adversarial samples, we employed two methods pertinent to our study. The first method, referred to as *UAP Adv.*, involves creating a universal perturbation that can mislead the model when applied to various inputs. The second method, termed *GAN Adv.*, uses a generative adversarial network to produce perturbations that consistently cause misclassification across different samples. Label noise was introduced by having some clients deliberately mislabel their data. This tested the system's resilience to incorrect labels introduced during training.

Despite these adversarial conditions, *LotusFL* maintained high accuracy and stable convergence across all four datasets. The MaxEnt-based watermark estimation effectively mitigated the impact of adversarial and dynamic watermarks by estimating and neutralizing their statistical properties. Additionally, the historical loss-based client selection mechanism identified clients with mislabeled data by their higher loss values, allowing the system to exclude their contributions from model updates.

TABLE VIII
PERFORMANCE OF *LotusFL* UNDER ADVERSARIAL CONDITIONS

Adv. Condition	MNIST	CIFAR-10	Tiny.	COVID-FL
Dynamic	.933±.004	.750±.038	.310±.035	.670±.080
UAP Adv.	.930±.005	.745±.040	.308±.036	.665±.082
GAN Adv.	.928±.006	.740±.042	.305±.038	.660±.085
Label Noise	.925±.007	.735±.045	.300±.040	.655±.088

6) *Complexity & Scalability:* The computational overhead of *LotusFL* mainly arises from two client-side operations: watermark estimation and batch adaptation. Our analysis shows these components introduce negligible overhead compared to baseline training while preserving scalability.

Computational Complexity Analysis. Watermark estimation requires solving M^2 entropy maximization subproblems for an $M \times M$ resolution mask, each involving $O(d)$ operations per data sample (where d denotes input dimensionality, e.g., image pixels). For a client with $|D_i|$ samples, this yields total complexity $O(M^2 \cdot d \cdot |D_i|)$. In comparison, one epoch of DNN training involves $O(L \cdot d \cdot |D_i| \cdot b)$ operations, where L represents layer count. Given $M^2 \ll L \cdot b$, watermark estimation is smaller than the per-epoch training overhead. Alg. 2 solves learning rate optimization in Line 12 via convex programming. Using precomputed Jacobian matrices reduces

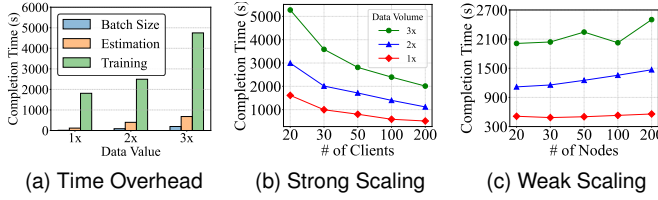


Fig. 6. Scalability and Overhead Result.

this to $O(1)$ per-client operations, as the quadratic form in Eq.(12) has closed-form solutions when $\bar{m}^r$ and $\bar{m}^{r+1}$ are orthogonal. The ξ -based batch scaling in Lines 10-11 ensures $O(1)$ arithmetic operations per client. *LotusFL* contributes less than 6% of per-epoch training overhead, as shown in Fig. 6a. The memory overhead introduced by *LotusFL* is also minimal. Client-side, the watermark estimator requires $O(M^2)$ for $\bar{m}$, a footprint typically much smaller than local model or data storage. Alg. 2 adds negligible $O(1)$ memory for storing a few scalar variables. Server-side, Alg. 1 incurs $O(N)$ memory to maintain historical statistics for each of the N clients, which is generally modest compared to storing the global model.

Scalability & Overhead. In large-scale deployments, the dominant $O(M^2 \cdot d \cdot |D_i|)$ watermark estimation cost remains client-local and parallelizable across devices. The server-side $O(N \log N)$ client selection complexity scales sub-linearly with typical FL client counts ($N \leq 10^4$). We evaluate the strong and weak scaling of decentralized sharding schedulers in *LotusFL*. Strong scaling tests *LotusFL*'s performance with increasing clients and fixed total function invocations, while weak scaling tests performance with increasing clients and fixed average invocations per node.

In strong scaling, we launch 500 clients, each invoking a function 100 times. Fig. 6b shows performance as *LotusFL* gradually increases data by one to three times, with completion time decreasing as client numbers increase.

In weak scaling, we set the average number of datasets per worker to 1 and evaluate performance with 200 concurrent invocations for 50 clients and 1000 for 200 clients. Fig. 6c shows that completion time does not rise significantly with increased workload intensity.

7) *Testbed Experiments with Edge Clients:* Fig. 3 illustrates our testbed setup for evaluating *LotusFL* with heterogeneous edge clients. Clients connected to switches via Ethernet, forming a LAN with the central server. FL data exchange (metadata and models) occurred via device IP addresses. Communication used sockets, with model transfer via scp. Hardware drivers were supported by Jetpack 5.1. Due to limited client device performance and memory, the Tiny-ImageNet task was excluded from testbed experiments.

Table IX presents the testbed accuracy and speedup of *LotusFL* compared to the benchmark. Compared to simulations, testbed accuracy slightly decreased; e.g., *LotusFL* dropped 0.46%-0.76% on CIFAR-10, a reasonable reduction. *LotusFL* still achieved the highest accuracy compared to the benchmark, surpassing other methods by 7.7%, particularly under watermarked inference and CIFAR-10 settings. For speedup, FedAvg was the $1 \times$ baseline; *LotusFL* achieved 1.43 to $2.12 \times$

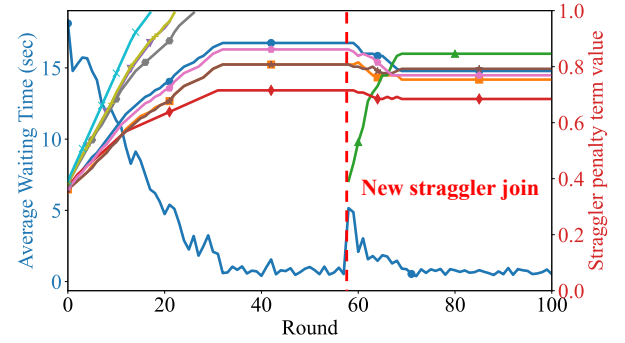


Fig. 7. Average waiting time of clients under *LotusFL* during straggler arrival, along with the change history of the straggler penalty term.

speedup.

We also evaluated dynamic performance under device heterogeneity and with stragglers. As shown in Fig. 7, the left Y-axis (blue line) is the average post-training synchronization waiting time. The right Y-axis is the straggler penalty from Alg. 1. Stragglers contributing more to waiting time receive heavier penalties. This penalty reduces client utility and increases its batch size, accelerating training. Lines show penalty term changes for different devices; as a device's penalty increases, its waiting time decreases. When a device's penalty reaches 1, its utility becomes zero, and it stops participating. A new straggler joined in Round 59, altering waiting time; its penalty is the green line. Device adjustments dynamically reduced waiting times, enhancing speedup.

V. RELATED WORK

Computational efficient federated learning Efficiency in FL is typically addressed by two main approaches: system-based and training-based methods [40]. System methods focus on properties like data non-IIDness or client utility, utilizing strategies like client selection [41], reweighting [42], and data sharing [23]. Client selection is challenged by system (e.g., computation, communication capability) and statistical (e.g., distributed, imbalanced, Non-IID data) heterogeneity [39]. Utility functions guide selection: statistical utility (sample or model-based) assesses client contribution [43]–[46], while system utility (time or deadline-based) identifies/avoids stragglers that prolong rounds [24], [47].

Training methods emphasize robust design or optimization to mitigate straggler impact [48]–[51]. These include adjusting batch sizes and learning rates. Batch size adjustment accelerates straggler convergence [14], [15], [52] or utilizes idle devices [53], improving overall FL efficiency.

Some works combine system and training methods for finer adjustments, enhancing FL efficiency or addressing specific challenges [54]–[56]. Our work adopts this interplay, motivated by how watermarked datasets can significantly impact the effectiveness of both client selection and batch size adjustment.

The impact of watermark in machine learning: Watermarks, as shortcut features in image data, demonstrably affect machine learning in visual tasks. Empirical research [4], [6],

TABLE IX
FL METHODS PERFORMANCE COMPARISON UNDER TESTBED.

Setting	Watermarked Dataset & Clean Inference						Watermarked Dataset & Inference					
	MNIST		CIFAR-10		COVID-FL		MNIST		CIFAR-10		COVID-FL	
	Acc.	Speed.	Acc.	Speed.	Acc.	Speed.	Acc.	Speed.	Acc.	Speed.	Acc.	Speed.
FedAvg	.944±.000	1.00±.00	.738±.002	1.00±.00	.698±.010	1.00±.00	.930±.000	1.00±.00	.566±.010	1.00±.00	.671±.011	1.00±.00
Oort	.964±.000	1.16±.01	.699±.006	1.17±.01	.670±.009	1.24±.02	.939±.000	0.88±.01	.564±.007	0.87±.05	.680±.010	1.23±.03
FedBalancer	.949±.000	1.20±.01	.750±.010	1.07±.01	.716±.008	1.15±.02	.906±.000	0.82±.01	.604±.002	0.79±.02	.688±.009	1.03±.02
PyramidFL	.966±.000	1.39±.02	.756±.013	1.35±.02	.735±.011	1.44±.03	.950±.000	1.20±.02	.759±.013	1.37±.02	.738±.012	1.50±.03
LotusFL	.973±.000	1.43±.04	.881±.012	2.11±.11	.762±.010	1.59±.04	.957±.000	1.44±.04	.867±.012	2.12±.10	.750±.011	1.65±.04

[57] indicates that both visible [6] and frequency domain [4] watermarks in training datasets impact model inference accuracy. Other studies theoretically explain watermark impacts on models [5], [21]. For instance, [5] shows watermarks in datasets can act as inadvertent cues affecting classification, likening this to the “Clever Hans” phenomenon where neural networks classify by watermark characteristics rather than actual visual features. Another study defines watermark features by their causal relationship to model predictions in medical X-ray image datasets [21].

Research also addresses problems caused by watermarks in model training. Approaches include watermark removal via data preprocessing [58], [59] or model-based methods [8], [60]. For instance, [60] uses adversarial training for robustness, while [8] fine-tunes models to ignore watermark-sensitive encodings in feature extraction layers. [28] considers watermarked datasets in FL, but focuses on training robustness and model accuracy. Our work addresses FL efficiency issues with watermarked datasets.

VI. CONCLUSION

We introduces *LotusFL*, a FL system tailored for training on watermarked data. Addressing the challenges posed by the diversity in watermark across FL participant devices, *LotusFL* efficiently facilitates FL without requiring prior knowledge of these watermark specifics. Our extensive experiments, conducted on a testbed of 40 Jetson devices. *LotusFL* improves the time-to-accuracy performance by $1.43 \sim 2.12 \times$ an accuracy by $1.35\% \sim 8.2\%$. We also conducted an ablation study of *LotusFL* to validate the contribution of each component to FL model performance in watermarked datasets.

REFERENCES

- [1] “A friendly federated ai framework.” [Online]. Available: <https://flower.ai/>
- [2] Google, “Robust and scalable tool for watermarking and identifying AI-generated images.” [Online]. Available: <https://deepmind.google/technologies/synthid>
- [3] NexGuard, “NexGuard Forensic Watermarking.” [Online]. Available: <https://nagra.vision/security-solutions/forensic-watermarking/>
- [4] S. Liu *et al.*, “Exploration of the Influence on Training Deep Learning Models by Watermarked Image Dataset,” in *ISPA/BDCloud/SocialCom/SustainCom*, 2021, pp. 421–428.
- [5] S. Lapuschkin *et al.*, “Unmasking Clever Hans Predictors and Assessing What Machines Really Learn,” *Nat. Commun.*, vol. 10, no. 1, p. 1096, 2019.
- [6] Y. Lyu, “Review of visible watermark’s influence on private information-related machine learning techniques,” *J. Phys.: Conf. Ser.*, vol. 1453, no. 1, p. 012070, 2020.
- [7] X. Luo *et al.*, “Rectifying the Shortcut Learning of Background for Few-Shot Learning,” in *NeurIPS*, vol. 34, 2021, pp. 13 073–13 085.
- [8] K. Bykov *et al.*, “Mark my words: Dangers of watermarked images in imagenet,” in *ECAI*. Springer, 2023, pp. 426–434.
- [9] P. Saranritichai *et al.*, “Overcoming Shortcut Learning in a Target Domain by Generalizing Basic Visual Factors from a Source Domain,” in *ECCV*, 2022, pp. 294–309.
- [10] M. Makar *et al.*, “Causally motivated shortcut removal using auxiliary labels,” in *AISTATS*, 2022, pp. 739–766.
- [11] Y. Xu *et al.*, “Enhancing federated learning with server-side unlabeled data by adaptive client and data selection,” *IEEE Trans. Mob. Comput.*, vol. 23, no. 4, pp. 2813–2831, 2024.
- [12] Y. Xu *et al.*, “Federated learning with client selection and gradient compression in heterogeneous edge systems,” *IEEE Trans. Mob. Comput.*, vol. 23, no. 5, pp. 5446–5461, 2024.
- [13] Q. Pan *et al.*, “Contextual client selection for efficient federated learning over edge devices,” *IEEE Trans. Mob. Comput.*, vol. 23, no. 6, pp. 6538–6548, 2024.
- [14] Z. Ma *et al.*, “Adaptive Batch Size for Federated Learning in Resource-Constrained Edge Computing,” *IEEE Trans. Mob. Comput.*, vol. 22, no. 1, pp. 37–53, 2023.
- [15] Q. Ye *et al.*, “DBS: Dynamic Batch Size For Distributed Deep Neural Network Training,” 2022.
- [16] T. B. Johnson and C. Guestrin, “Training Deep Models Faster with Robust, Approximate Importance Sampling,” in *NeurIPS*, vol. 31, 2018.
- [17] A. F. Qasim *et al.*, “Digital watermarking: Applicability for developing trust in medical imaging workflows state of the art review,” *Comput. Sci. Rev.*, vol. 27, pp. 45–60, 2018.
- [18] W. Zheng *et al.*, “Robust and high capacity watermarking for image based on DWT-SVD and CNN,” in *ICIEA*, 2018, pp. 1233–1237.
- [19] J. Nin and S. Ricciardi, “Digital Watermarking Techniques and Security Issues in the Information and Communication Society,” in *AINA Workshops*, 2013, pp. 1553–1558.
- [20] R. Geirhos *et al.*, “Shortcut learning in deep neural networks,” *Nat. Mach. Intell.*, vol. 2, no. 11, pp. 665–673, 2020.
- [21] D. Bellamy *et al.*, “A structural characterization of shortcut features for prediction,” *Eur. J. Epidemiol.*, vol. 37, no. 6, pp. 563–568, 2022.
- [22] F. Lai *et al.*, “Oort: Efficient federated learning via guided participant selection,” in *OSDI*, 2021, pp. 19–35.
- [23] J. Shin *et al.*, “FedBalancer: Data and pace control for efficient federated learning on heterogeneous clients,” in *MobiSys*, ser. MobiSys ’22, 2022, pp. 436–449.
- [24] C. Li *et al.*, “PyramidFL: A fine-grained client selection framework for efficient federated learning,” in *MobiCom*, ser. MobiCom ’22. New York, NY, USA: Association for Computing Machinery, Oct. 2022, pp. 158–171.
- [25] L. Miao *et al.*, “A maximum entropy approach to unsupervised mixed-pixel decomposition,” *IEEE Trans. Image Process.*, vol. 16, no. 4, pp. 1008–1021, 2007.
- [26] W. Li and Q. Guo, “A maximum entropy approach to one-class classification of remote sensing imagery,” *Int. J. Remote Sens.*, vol. 31, no. 8, pp. 2227–2235, 2010.
- [27] S. Voloshynovskiy *et al.*, “A Stochastic Approach to Content Adaptive Digital Image Watermarking,” in *IH*, Berlin, Heidelberg, 2000, pp. 211–236.
- [28] T. Ling *et al.*, “Federated morozov regularization for shortcut learning in privacy preserving learning with watermarked image data,” in *ACM MM*, 2024, pp. 4899–4908.
- [29] P. Zhao and T. Zhang, “Stochastic Optimization with Importance Sampling for Regularized Loss Minimization,” in *ICML*, 2015, pp. 1–9.

- [30] A. Katharopoulos and F. Fleuret, "Not All Samples Are Created Equal: Deep Learning with Importance Sampling," *arXiv preprint arXiv:1803.00942*, 2019.
- [31] S. Theodoridis, *Machine Learning: A Bayesian and Optimization Perspective*, Amsterdam Boston Heidelberg London New York Oxford Paris San Diego San Francisco Singapore Sydney Tokyo, 2015.
- [32] S. Song *et al.*, "Learning from Data with Heterogeneous Noise using SGD," in *AISTATS*, 2015, pp. 894–902.
- [33] Ansible, "Ansible is the simplest way to automate apps and it infrastructure." [Online]. Available: <https://www.ansible.com>
- [34] P. Chrabaszcz *et al.*, "A downsampled variant of imagenet as an alternative to the cifar datasets," *arXiv preprint arXiv:1707.08819*, 2017.
- [35] R. Yan *et al.*, "Label-Efficient Self-Supervised Federated Learning for Tackling Data Heterogeneity in Medical Imaging," *IEEE Trans. Med. Imaging*, vol. 42, no. 7, pp. 1932–1943, 2023.
- [36] M. Begum and M. S. Uddin, "Towards the development of an effective image watermarking system," *Secur. Priv.*, vol. 5, no. 2, p. e196, 2022.
- [37] B. McMahan *et al.*, "Communication-Efficient Learning of Deep Networks from Decentralized Data," in *AISTATS*, 2017, pp. 1273–1282.
- [38] S. V. Voloshynovskiy *et al.*, "Generalized watermarking attack based on watermark estimation and perceptual remodulation," in *Proc. SPIE Electron. Imaging*, San Jose, CA, 2000, pp. 358–370.
- [39] L. Fu *et al.*, "Client Selection in Federated Learning: Principles, Challenges, and Opportunities," *IEEE Internet Things J.*, 2023.
- [40] C. Yang *et al.*, "Flash: Heterogeneity-aware federated learning at scale," *IEEE Trans. Mob. Comput.*, vol. 23, no. 1, pp. 483–500, 2022.
- [41] T. Tuor *et al.*, "Overcoming Noisy and Irrelevant Data in Federated Learning," in *ICPR*, 2021, pp. 5020–5027.
- [42] A. Li *et al.*, "Sample-level Data Selection for Federated Learning," in *INFOCOM*, 2021, pp. 1–10.
- [43] H. Cao *et al.*, "Birds of a feather help: Context-aware client selection for federated learning," in *TVAFL @ AAAI*, 2022, pp. 1–8.
- [44] L. Wang *et al.*, "Cmf1: Mitigating communication overhead for federated learning," in *ICDCS*. IEEE, 2019, pp. 954–964.
- [45] P. Zhou *et al.*, "Are you left out? an efficient and fair federated learning for personalized profiles on wearable devices of inferior networking conditions," *Proc. ACM Interact. Mob. Wearable Ubiquitous Technol. (IMWUT)*, vol. 6, no. 2, pp. 1–25, 2022.
- [46] M. M. Amiri *et al.*, "Convergence of update aware device scheduling for federated learning at the wireless edge," *IEEE Trans. Wireless Commun.*, vol. 20, no. 6, pp. 3643–3658, 2021.
- [47] T. Nishio and R. Yonetani, "Client selection for federated learning with heterogeneous resources in mobile edge," in *ICC*. IEEE, 2019, pp. 1–7.
- [48] N. Rodríguez-Barroso *et al.*, "Survey on federated learning threats: Concepts, taxonomy on attacks and defences, experimental study and challenges," *Inf. Fusion*, vol. 90, pp. 148–173, 2023.
- [49] T. Li *et al.*, "Federated Optimization in Heterogeneous Networks," *Proc. Mach. Learn. Syst. (MLSys)*, vol. 2, pp. 429–450, 2020.
- [50] X. Zhang *et al.*, "FedPD: A Federated Learning Framework With Adaptivity to Non-IID Data," *IEEE Trans. Signal Process.*, vol. 69, pp. 6055–6070, 2021.
- [51] T. Li *et al.*, "FedDANE: A Federated Newton-Type Method," in *Proc. Asilomar Conf. Signals Syst. Comput.*, 2019, pp. 1227–1231.
- [52] B. Liu *et al.*, "Accelerate Mini-batch Machine Learning Training With Dynamic Batch Size Fitting," in *IJCNN*, 2019, pp. 1–8.
- [53] D. Shi *et al.*, "To Talk or to Work: Dynamic Batch Sizes Assisted Time Efficient Federated Learning Over Future Mobile Edge Devices," *IEEE Trans. Wireless Commun.*, vol. 21, no. 12, pp. 11 038–11 050, 2022.
- [54] W. Liu *et al.*, "Dynamite: Dynamic interplay of mini-batch size and aggregation frequency for federated learning with static and streaming dataset," *IEEE Trans. Mob. Comput.*, 2023.
- [55] Y. Xu *et al.*, "Federated learning with client selection and gradient compression in heterogeneous edge systems," *IEEE Trans. Mob. Comput.*, 2023.
- [56] Y. Xu *et al.*, "Adaptive control of local updating and model compression for efficient federated learning," *IEEE Trans. Mob. Comput.*, vol. 22, no. 10, pp. 5675–5689, 2022.
- [57] S. Hong *et al.*, "Poster: On the Feasibility of Training Neural Networks with Visibly Watermarked Dataset," 2019.
- [58] Y. Liu *et al.*, "WDNet: Watermark-Decomposition Network for Visible Watermark Removal," in *WACV*, 2021, pp. 3685–3693.
- [59] T. Dekel *et al.*, "On the Effectiveness of Visible Watermarks," in *CVPR*, 2017, pp. 6864–6872.
- [60] N. M. Müller *et al.*, "Localized Shortcut Removal," in *CVPR*, 2023, pp. 3721–3725.



Tao Ling received his B.S. degree in Internet of Things engineering from Heilongjiang University in 2019, and her M.S. degree in computer science from the University of Birmingham in 2020. Now he is currently a Ph.D student in Computer Science from the Hong Kong Polytechnic University. His research interests include edge computing, federated learning, and watermark.



Siping Shi (Graduate Student Member, IEEE) received her B.S. degree in computer science from Sichuan University in 2014 and her M.S. degree in computer applied technology from the University of Chinese Academy of Sciences in 2017, and her Ph.D degree in Computer Science from The Hong Kong Polytechnic University in 2023. She is currently a Postdoctoral Fellow at The Hong Kong Polytechnic University. Her research interests include edge computing, federated learning and analytics, and privacy-preserving vision foundation models.



Hao Wang (Member, IEEE) received the BE degree in information security and the ME degree in software engineering from Shanghai Jiao Tong University, Shanghai, China, in 2012 and 2015 respectively, and the PhD degree from the Department of Electrical and Computer Engineering, the University of Toronto, Canada in 2020. He is an assistant professor with the Department Electrical and Computer Engineering, Stevens Institute of Technology, Hoboken, NJ, USA. His research interests include distributed ML systems, AI security and forensics, privacy-preserving data analytics, serverless computing, and high-performance computing. He is a recipient of the NSF CRII Award.



Chuang Hu received his BS and MS degrees from Wuhan University in 2013 and 2016. He received his Ph.D. degree from the Hong Kong Polytechnic University in 2019. He is currently an Associate Researcher in the School of Computer Science at Wuhan University. His research interests include edge learning, federated learning/analytics, and distributed computing.



Dan Wang (Senior Member, IEEE) is currently a professor with the Department of Computing, The Hong Kong Polytechnic University. His research interests lie in networked systems, and recently in the inter-discipline domains of smart energy systems. He publishes in ACM SIGCOMM, ACM SIGMETRICS, IEEE INFOCOM, and in inter-discipline conferences, such as ACM e-Energy, ACM Buildsys. He won the Best Paper Award of ACM e-Energy 2018 and the Best Paper Award of ACM Buildsys 2018. He is currently the steering committee chair of IEEE/ACM IWQoS and the steering committee chair of ACM e-Energy. He is an advisor of EMSD, the Hong Kong SAR government. He has extensive experience in applying his research results to industry, including Huawei, IBM, Henderson, etc. He won the TechConnect Global Innovation Award, in 2017.